

Proyecto Final — System Health Pipeline

Joaquín Puente

Debido a la configuración de entra que solo permite PDF, se adjunta enlace de github repositorio público donde puede ser validado que todo el contenido es subido antes de la fecha de entrega:

<https://github.com/Its-Japo/ProyectoFinalScripting>

1. Arquitectura

El sistema es un **pipeline lineal de tres fases**, donde la salida de cada fase es la entrada de la siguiente:

1. **Fase 1 — Recolección (Bash, 01_recoleccion.sh)**: recolecta las métricas del sistema (CPU, RAM y disco) y las escribe en un archivo CSV clave/valor (`metrics_raw.csv`).
2. **Fase 2 — Procesamiento (PowerShell, 02_procesamiento.ps1)**: toma ese CSV, limpia y normaliza los datos, evalúa cada métrica contra sus umbrales y produce un reporte estructurado en formato JSON (`report.json`).
3. **Fase 3 — Notificación (Python, 03_notificacion.py)**: lee el JSON y, según la criticidad detectada, envía una alerta al canal de Discord.

En resumen, el flujo de datos es: `01_recoleccion.sh` → `metrics_raw.csv` → `02_procesamiento.ps1` → `report.json` → `03_notificacion.py` → Discord.

Fase	Tecnología	Función	Entrada	Salida
1. Recolección	Bash	Extrae métricas (CPU, RAM, disco)	—	<code>data/metrics_raw.csv</code>
2. Procesamiento	PowerShell (pwsh)	Limpia datos y evalúa umbrales	<code>metrics_raw.csv</code>	<code>data/report.json</code>
3. Notificación	Python 3	Envía alerta según criticidad	<code>report.json</code>	Mensaje en Discord

2. Estructura de archivos

```
Proyecto_scripting/
├── 01_recoleccion.sh      # Fase 1 – recolección (Bash)
├── 02_procesamiento.ps1  # Fase 2 – análisis y umbrales (PowerShell)
├── 03_notificacion.py     # Fase 3 – notificación a Discord (Python)
├── run_pipeline.sh        # Orquestador: corre las 3 fases en orden
├── config.env             # Webhook de Discord + umbrales
└── README.md             # Este documento
```

```
└─ data/
   └─ metrics_raw.csv      # Salida de la Fase 1 (ejemplo incluido)
      └─ report.json       # Salida de la Fase 2 (ejemplo incluido)
```

3. Contrato de datos entre fases

Fase 1 → Fase 2: data/metrics_raw.csv

CSV en bruto, una fila por métrica:

```
metric,value,unit,timestamp,host
cpu_usage,12.5,percent,2026-07-05T21:40:00-06:00,Debian
mem_used_pct,36.4,percent,2026-07-05T21:40:00-06:00,Debian
disk_used_pct,42,percent,2026-07-05T21:40:00-06:00,Debian
...
```

Fase 2 → Fase 3: data/report.json

Reporte estructurado con el estado de cada métrica y el estado general:

```
{
  "hostname": "Debian",
  "collected_at": "2026-07-05T21:40:00-06:00",
  "generated_at": "2026-07-05T21:40:01.123",
  "overall_status": "OK",
  "metrics": [
    {
      "name": "disk",
      "label": "Disk Usage (/)",
      "value": 42,
      "unit": "percent",
      "status": "OK",
      "threshold_warning": 80,
      "threshold_critical": 90,
      "message": "Disk Usage (/): 42percent -> OK"
    }
  ],
  "raw": { "...": "valores crudos de todas las métricas" }
}
```

4. Umbrales definidos

Se configuran en `config.env` (porcentaje de uso). Regla: valor $\geq$ CRIT → **CRITICAL**; valor $\geq$ WARN → **WARNING**; en otro caso **OK**. El `overall_status` es el **peor** estado entre las tres métricas.

Métrica	WARNING ($\geq$)	CRITICAL ($\geq$)
CPU	75 %	90 %

Métrica	WARNING (≥)	CRITICAL (≥)
Memoria	80 %	90 %
Disco	80 %	90 %

5. Requisitos

- Debian con **Bash**, **PowerShell (pwsh)** y **Python 3** instalados.
- **curl** (para instalar/usar) — el envío a Discord usa la librería estándar de Python (`urllib`), sin dependencias `pip`.
- Un **webhook de Discord** configurado en `config.env`.

6. Configuración del webhook de Discord

1. En Discord, abrir el canal donde se desean recibir las métricas.
2. **Editar canal** → **Integraciones** → **Webhooks** → **Nuevo webhook**.
3. Pega la URL en `config.env`:

```
DISCORD_WEBHOOK_URL="https://discord.com/api/webhooks/XXXX/YYYY"
```

7. Ejecución

Pipeline completo

```
./run_pipeline.sh
```

Prueba sin enviar a Discord (dry-run)

Ejecuta las fases 1 y 2 reales y muestra el mensaje que *se enviaría*, sin mandarlo:

```
./run_pipeline.sh --dry-run
```

Fase por fase (manual)

```
bash 01_recoleccion.sh           # genera data/metrics_raw.csv
pwsh -NoProfile -File 02_procesamiento.ps1 # genera data/report.json
python3 03_notificacion.py        # envía la alerta a Discord
python3 03_notificacion.py --dry-run # muestra el payload sin enviar
```